

TETRIS EDUCATIVO

Sistemas Operativos

Integrantes:

González Carmona Adriel – 010211972

Soto Terán Azul Fernanda – 010212224

Plata Hernández Leslie Berenice – 190219329

Carmona Orea Lizbeth – 010212016

Gutiérrez Guillén Román – 010211992

Docente

Ríos Olivares César Antonio



LAUREATE INTERNATIONAL UNIVERSITIES®

Descripción general del proyecto

Tetris Educativo es una adaptación pedagógica del clásico videojuego Tetris, concebida como recurso didáctico para el reforzamiento de habilidades aritméticas básicas en niñas, niños y adolescentes. El proyecto conserva la mecánica original de manipular tetraminós para conformar líneas completas, pero incorpora una capa de evaluación cognitiva mediante un sistema de progresión por niveles condicionado al desempeño matemático del jugador.

Cada nivel exige completar diez líneas; al alcanzar este objetivo, el jugador debe resolver correctamente una operación aritmética para habilitar el avance. La dificultad escala de forma incremental: el nivel 1 plantea sumas de un dígito, el nivel 2 restas de un dígito, el nivel 3 multiplicaciones de un dígito y el nivel 4 divisiones de un dígito. Esta estructura articula los principios de aprendizaje progresivo (Zona de Desarrollo Próximo de Vygotsky) con la teoría del flujo cognitivo (Csíkszentmihályi), generando un entorno en el que el reto perceptual-motor del Tetris se combina con la práctica deliberada del cálculo mental. El proyecto incluye retroalimentación inmediata, registro de aciertos y métricas de rendimiento, útiles tanto para la autoevaluación del jugador como para el diagnóstico docente.

Justificación

Tetris Educativo se vincula de manera prioritaria al ODS 4 (Educación de Calidad), en particular a las metas 4.1 (resultados de aprendizaje pertinentes y efectivos en educación básica), 4.6 (alfabetización y nociones elementales de aritmética para toda la población) y 4.a (creación de entornos de aprendizaje inclusivos y seguros mediante TIC). Al integrar contenido matemático dentro de una mecánica de juego ampliamente reconocida, el proyecto reduce la ansiedad matemática y favorece la práctica sostenida del cálculo, dos factores asociados a la mejora del rendimiento escolar. Asimismo, contribuye al ODS 10 (Reducción de las Desigualdades), meta 10.3, al ofrecer un recurso digital de libre acceso que complementa la enseñanza formal y atenúa brechas de aprendizaje vinculadas al contexto socioeconómico. Finalmente, se alinea con el ODS 9 (Industria, Innovación e Infraestructura), meta 9.c, al aprovechar el desarrollo de software y las TIC para diseñar soluciones educativas innovadoras y replicables.

Objetivo del proyecto y alcance funcional

Objetivo general: Programar una aplicación educativa funcional que ayude a que niños refuercen sus matemáticas básicas mientras se divierten.

Objetivos específicos:

- **Mecánicas base de Tetris mediante gestión de memoria:** Desarrollar subrutinas para la asignación pseudoaleatoria de datos, rotación de matrices

lógicas, rutinas de detección de colisiones espaciales y reestructuración automatizada de buffers de memoria al purgar filas llenas.

- **Módulo de operaciones e interrupciones:** Implementar un subsistema de procesamiento aritmético segmentado en cuatro niveles de complejidad que condicione la ejecución del proceso principal mediante filtros lógicos de control (Nivel 1: Sumas; Nivel 2: Restas; Nivel 3: Multiplicaciones; Nivel 4: Divisiones).
- **Gestión de la interfaz (UI) en tiempo real:** Controlar los descriptores visuales que muestran las variables globales de puntuación, el nivel de ejecución activo y la cola (*queue*) de la siguiente pieza, emitiendo señales visuales de interrupción en función del estado de la validación de entrada.
- **Orquestación del flujo de navegación:** Diseñar e intercomunicar los distintos contextos de ejecución de la interfaz que conforman el software (Menú de inicio, Instrucciones, Formulario de cuestionamiento, Pantallas finales de término de proceso).

Alcance: El sistema opera de manera estable y local tanto en entornos de escritorio (Windows, Linux, macOS) como en sistemas operativos móviles (Android). A través del motor de compilación de Flet.

Tecnologías, herramientas y requerimientos del software

- **Lenguaje de programación:** Python 3.13.3.
- **Framework principal:** Flet.
- **Sistemas compatibles:** Windows, Linux, macOS y Android.
- **Hardware mínimo recomendado:** Computadora de escritorio o dispositivo móvil básico equipado con 4 GB de memoria RAM (capaz de albergar el runtime de Python y los subprocesos de la interfaz).
- **Entorno de desarrollo integrado (IDE):** Visual Studio Code.

Arquitectura del sistema y diseño general

El sistema sigue una arquitectura modular enfocada en la separación de responsabilidades y en la gestión de procesos independientes:

- **Capa UI (Presentación):** Funciones lógicas encargadas de interactuar con el gestor de ventanas del sistema operativo. Estas funciones contralan lo que el usuario ve (`page.controls.clear()`) y proyectan las cosas que al usuario le gustaría ver (`ft.Stack`, `ft.Container`, `ft.TextField`, `ft.Row`, `ft.Column`). Las vistas principales de la aplicación son: `show_main_menu`, `show_instructions`, `show_question_screen`, `show_victory_screen` y `show_game_over_screen`.

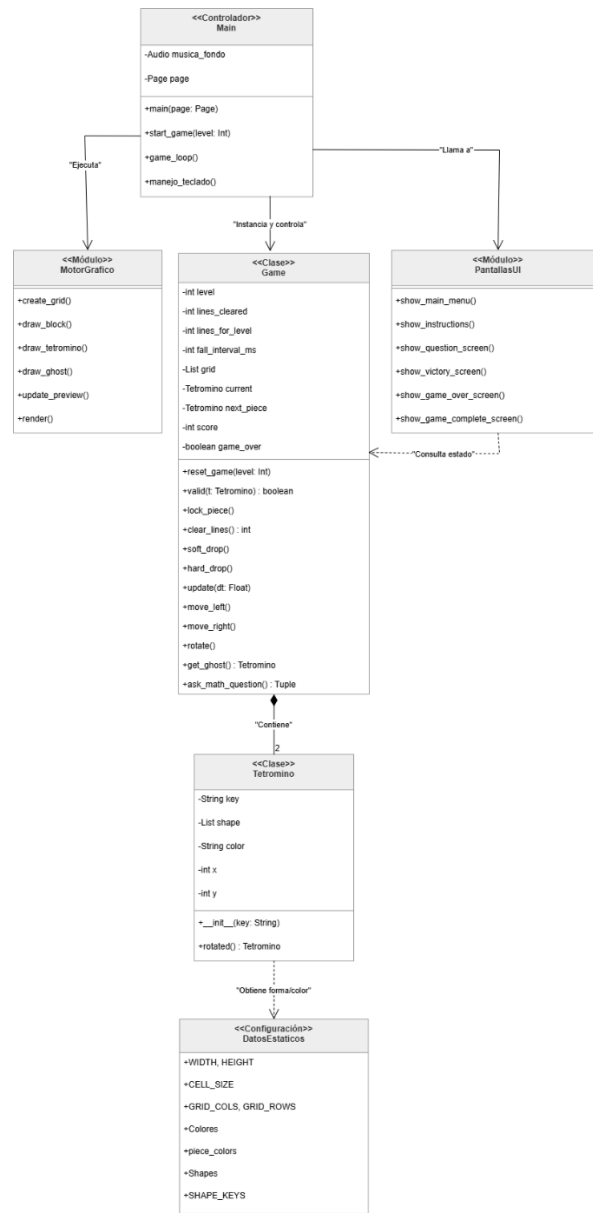
- **Capa Lógica (Módulo Game):** Conjunto de funciones y estructuras de control que administran el estado físico de la partida. El cual actualiza las variables de puntuación y manipula el intervalo del temporizador (`fall_interval_ms`).
- **Módulo de Datos (Módulo Tetromino):** Estructura encargada de la forma de las figuras (I, O, T, J, L, S, Z).
- **Módulo Educativo:** Rutinas diseñadas para la generación pseudoaleatoria de operaciones aritméticas adaptativas (`ask_math_question`). Actúan como filtros que bloquean temporalmente el avance del juego mediante pantallas, para después confirmar los resultados (`on_correct`, `on_wrong`) de acuerdo con los números ingresados.

Tecnologías, frameworks, librerías y versiones utilizadas

La selección del stack tecnológico se basó en la adecuación técnica para el control de concurrencia y la eficiencia en la respuesta ante interrupciones de Entrada/Salida en tiempo real:

- Lenguaje Base: Python 3.13.3. Se eligió por su sintaxis de alto nivel que optimiza el Desarrollo Rápido de Aplicaciones (RAD) y por su robusta biblioteca estándar para el manejo de asincronía, permitiendo enfocar el esfuerzo en el diseño de los algoritmos de control de la matriz de datos.
- Framework Core: Flet. Elemento de software que funciona como una capa de abstracción sobre el subsistema gráfico del sistema operativo. Su motor reactivo optimiza el rendimiento del procesador mediante actualizaciones parciales de los componentes en memoria con el método `page.update()`, basándose estrictamente en las constantes lógicas de la grilla (`CELL_SIZE`, `GRID_COLS`), lo que garantiza una latencia mínima ante los eventos de entrada del teclado o pantallas táctiles.
- Módulos Estándar Complementarios:
 - * `asyncio`: Módulo crítico integrado para la orquestación asíncrona del bucle de juego (`game_loop`), administrando la asignación de recursos y tiempos de ejecución de las tareas en segundo plano.
 - `random`: Utilizado para la generación estocástica de los bloques de datos (Tetrominos) y para la parametrización dinámica de los operandos del módulo de evaluación.

Diagrama UML



Justificación técnica de selección de herramientas

La selección de las herramientas se fundamentó en la viabilidad técnica para cumplir tanto con los requisitos funcionales del software, como con las diseños.

1. Justificación de Python

El lenguaje Python fue seleccionado por su alta portabilidad y por incluir en su biblioteca estándar módulos avanzados orientados al control de subprocessos concurrentes. Esto facilitó modelar el juego mediante bloques de código modulares y estructuras de datos limpias, encapsulando las variables de estado para evitar condiciones de carrera o colisiones de memoria en el hilo de ejecución. Su sintaxis de

alto nivel optimizó el desarrollo del software, abstrayendo la gestión manual de punteros.

2. Justificación de Flet

Flet se seleccionó como el entorno de ejecución de la interfaz debido a su capacidad para operar como una capa intermedia eficiente que interactúa de manera óptima con los subsistemas de los sistemas operativos actuales:

- **Abstracción Gráfica y Renderizado Reactivo:** Flet actúa como una interfaz de abstracción que delega el procesamiento visual al motor multiplataforma subyacente (Flutter).
- **Orquestación Asíncrona (Control de Temporización):** A través de la integración nativa del módulo `asyncio`, el sistema administra un bucle de ejecución concurrente que prescinde de la creación de hilos pesados (*threads*) de núcleo para cada tarea, evitando la sobrecarga por cambio de contexto en el planificador del sistema operativo.
- **Manejo de Eventos Simultáneos y Entrada/Salida (I/O) no Bloqueante:** A diferencia de las librerías de software tradicionales que bloquean el flujo global para capturar la entrada estándar, Flet implementa una escucha asíncrona de interrupciones de E/S mediante el control de captura de texto `ft.TextField`.
- **Portabilidad y Abstracción del Kernel Multiplataforma:** La principal ventaja estructural de esta herramienta radica en su capacidad para traducir las directrices del programa en llamadas al sistema nativas de múltiples plataformas. La misma lógica programada en Python interactúa de manera transparente con los gestores de ventanas en entornos de escritorio (Windows, Linux, macOS) y se compila sin alteraciones funcionales para operar sobre el kernel de dispositivos móviles (Android).

3. Conclusión de la selección

La integración de Python y Flet representó una arquitectura de software óptima y robusta para la gestión de recursos del sistema. Esta infraestructura permitió codificar la lógica del juego (control de buffers bidimensionales y validación de colisiones vectoriales) en paralelo con un subsistema de Entrada/Salida asíncrono.

Modelo y estructuras de datos internos

1. Estructura de Datos Central: El Tablero (`self.grid`)

El componente de datos más crítico de la aplicación es la matriz del tablero de juego, mapeada directamente en memoria con dimensiones fijas de 10 columnas por 20 filas.

- **Tipo de Dato:** Esta estructura consiste en una lista de listas indexada (`List[List[str or None]]`).
- **Propósito:** Funciona como un buffer de almacenamiento permanente que registra el estado físico de los bloques de datos una vez consolidados en la cuadrícula.
- **Contenido de Celda:** Cada dirección de la matriz almacena un valor `None` si la posición está disponible en el espacio de memoria, o una cadena de texto en formato hexadecimal (ej. `"#ff0000"`) que define el identificador cromático del bloque fijo. Esto permite que el motor de renderizado asocie y dibuje directamente el control en pantalla basándose en el contenido de la matriz.

2. Descriptor de Datos Activo (Módulo Tetromino)

El script gestiona un modelo de datos estructurado para administrar las variables de la pieza en ejecución activa (`current_piece`) y la pieza en cola de espera (`next_piece`).

- **Matriz de Forma (shape):** Matriz lógica bidimensional que mapea la geometría específica del bloque (I, O, T, J, L, S, Z). Las banderas con valor 1 indican un bit de ocupación activo, mientras que los valores 0 representan espacio libre.
- **Coordenadas de Posicionamiento (x, y):** Variables de tipo entero (`int`) que guardan la posición de anclaje de la pieza dentro de los índices de la matriz general del tablero. Estas variables determinan la ubicación absoluta utilizada por el hilo del sistema para validar colisiones de desbordamiento.
- **Descriptor Cromático:** Cadena de caracteres hexadecimal mapeada según el tipo de bloque, utilizada por las rutinas de visualización de Flet y por la subrutina de fijación de bloques para escribir el color final en el buffer del tablero.

3. Estructuras de Control de Estado y Temporización

Variables de control numérico que administran los subprocesos lógicos, la carga de dificultad y el flujo del programa:

- **self.score:** Variable acumulativa encargada de almacenar las métricas de rendimiento del usuario.
- **self.level:** Indicador de estado que determina el contexto de dificultad y la complejidad del filtro de operaciones matemáticas del sistema.
- **self.fall_interval_ms:** Umbral de tiempo medido en milisegundos que regula el ciclo del temporizador de descenso. Esta variable se actualiza dinámicamente y

decrece con cada salto de nivel para exigir una mayor velocidad de respuesta al usuario.

- **self.fall_timer:** Acumulador de tiempo de ejecución que mide los milisegundos transcurridos entre ciclos.

4. Modelo de Datos Educativos

La lógica de las preguntas aritméticas utiliza un esquema de generación algorítmica en tiempo de ejecución, prescindiendo de una base de datos o almacenamiento secundario estático:

- **Generación de Operaciones:** La función `ask_math_question()` invoca al módulo `random` para procesar los números en la memoria intermedia de forma dinámica. El sistema aplica filtros lógicos para adecuar la complejidad aritmética al nivel de ejecución actual (por ejemplo, en el Nivel 2 restringe los operandos para que el sustraendo no cause un resultado negativo inválido para el entorno educativo).
- **Buffer de Entrada:** Los datos introducidos por el usuario en la interfaz se gestionan mediante el componente interactivo `ft.TextField`. El hilo del programa almacena la entrada como una cadena de texto (`str`), restringe la longitud a un máximo de 3 caracteres y aplica una validación de dígitos antes de hacer el moldeado a un número entero para la verificación lógica.

Estándares de codificación y convenciones de nombres

1. Convenciones de Nomenclatura

Para asegurar la legibilidad del código y facilitar el mantenimiento del software, se implementaron reglas consistentes de etiquetado en todos los módulos lógicos:

- **Estructuras Principales y Módulos:** Se utiliza la convención `PascalCase` para definir los constructores lógicos de datos como `Game` y `Tetromino`, respetando el estándar de legibilidad de Python.
- **Funciones y Rutinas de Control:** Se emplea estrictamente la convención `snake_case` (letras minúsculas unidas por guiones bajos). Esta regla se aplica a todas las funciones operativas del sistema (ej. `clear_lines`, `show_question_screen`, `ask_math_question`).
- **Variables y Descriptores Locales:** Se mapean en formato `snake_case` (ej. `lines_cleared`, `fall_timer`) para mantener la homogeneidad con los nombres de las funciones operativas.
- **Constantes de Configuración Global:** Todas las variables inmutables que definen parámetros fijos del sistema se declaran en `UPPER_SNAKE_CASE`.

Esto incluye la paleta cromática en hexadecimal (ej. DARK_BLUE, RED) y las dimensiones físicas del viewport (ej. GRID_ROWS, CELL_SIZE).

2. Estándares de Estructura y Diseño de Software

- **Separación de Responsabilidades (SoC):** El software aísla conceptualmente el motor lógico de procesamiento (administrado en el módulo Game) de la capa de visualización encargada de interactuar con el gestor de ventanas del sistema operativo, garantizando que las reglas del juego no queden acopladas rígidamente al entorno gráfico.
- **Encapsulación de Datos:** Las estructuras empaquetan internamente sus propias variables de estado. Por ejemplo, el bloque Tetromino retiene el control de su matriz binaria (shape) y computa sus desplazamientos de forma autónoma sin permitir modificaciones corruptas por parte del hilo externo.
- **Uso de Constantes Globales:** Centralizar las dimensiones lógicas y colores en el bloque inicial de constantes facilita la portabilidad del código, ya que cualquier modificación en CELL_SIZE escala y reconfigura de forma automática todas las rutinas de visualización del sistema.
- **Documentación Técnica Integrada:** Se incluyeron comentarios detallados en las rutinas de mayor complejidad algorítmica (como el bucle de validación espacial de colisiones y la purga de buffers al eliminar filas) para detallar el flujo de datos y acelerar las fases de depuración (*debugging*).

Estructura del proyecto (carpetas, módulos, dependencias)

El proyecto se diseñó para operar de forma independiente y contenida dentro de un único script ejecutable, simplificando la distribución del software en los diferentes sistemas operativos. Sin embargo, mantiene una clara separación de tareas a nivel de arquitectura lógica interna.

1. Estructura Física

- **Directorio Raíz:** Contiene el archivo principal de ejecución (tetrisflet.py) y un archivo de documentación de inicio rápido (README.md).

2. Modularidad Lógica Interna

El script se encuentra estructurado internamente en bloques de código independientes que se reparten las tareas del sistema:

- **Bloque de Configuración Basal:** Define las variables globales del sistema, dimensiones de la matriz de memoria (GRID_COLS, GRID_ROWS), tamaño de

los contenedores gráficos y el diccionario con las matrices binarias de los bloques.

- **Módulo de Datos de Bloque (class Tetromino):** Estructura encargada de registrar el estado físico de la pieza activa en memoria, almacenando sus matrices de bits, posición y control de rotación.
- **Módulo de Procesamiento Central (class Game):** Actúa como el controlador principal de recursos. Valida las colisiones espaciales, coordina la fijación de datos en la matriz y procesa los cálculos de puntuación del juego.
- **Capa de Interfaz de Usuario (UI):** Grupo de funciones asíncronas encargadas de interactuar con la API de Flet para limpiar, redibujar y desplegar las ventanas sobre el sistema operativo (menús, paneles interactivos y formularios de texto).
- **Módulo Educativo:** Centraliza las rutinas de temporización y cálculo de hilos para procesar las operaciones aritméticas y desviar el flujo del software hacia los callbacks correspondientes.

3. Dependencias del Entorno

- **Framework Core (Externa):** Flet. Es la dependencia fundamental del proyecto, encargada de proveer la capa de abstracción para comunicar el código de Python con el motor multiplataforma de rendering gráfico.
- **Librerías Estándar (Nativas de Python):** Se utiliza `asyncio` para coordinar de forma concurrente el bucle de eventos sin bloquear los hilos de Entrada/Salida del sistema operativo, y `random` para los procesos estocásticos de selección de piezas y números.

Lógica de funcionamiento del juego

1. Bucle Principal (game_loop)

- El proceso central de la aplicación corre de manera continua dentro de una corrutina administrada por el despachador de tareas de `asyncio`.
- Reemplaza el control imperativo de fotogramas fijos por retardos asíncronos programados (`asyncio.sleep(0.05)`), liberando ciclos de la CPU mientras no se requieran actualizaciones.
- Captura e interrumpe los canales de Entrada/Salida para procesar las llamadas del teclado o eventos táctiles. Si el juego detecta la bandera `game_over`, interrumpe el ciclo lógico e invoca la rutina visual `show_game_over_screen()`; de lo contrario, delega la actualización de la física al método `game.update(50)` y refresca el árbol de componentes del sistema operativo.

2. Descenso Automático y Temporización

- Al inicializar o restablecer el estado de la partida, el sistema evalúa la variable `fall_interval_ms`, calculando una velocidad de caída que disminuye progresivamente con cada incremento de nivel para exigir mayor velocidad de procesamiento al hilo de juego.
- El método `update(50)` incrementa la variable interna de control de tiempo. En el momento en que este acumulador sobrepasa el umbral establecido en `fall_interval_ms`, el sistema ordena el desplazamiento forzado de la estructura de datos una unidad hacia abajo en el eje Y.

3. Validación de Colisiones

- La rutina de control comprueba vectorialmente que los bits activos del bloque en movimiento no excedan los límites indexados de la matriz de memoria.
- Del mismo modo, inspecciona que las coordenadas de destino no apunten a índices que ya contengan cadenas de color hexadecimal registradas en el buffer de `self.grid`. Si la rutina detecta una colisión al intentar realizar un desplazamiento vertical, interrumpe el movimiento e invoca inmediatamente la función `lock_piece()`.

4. Consolidación de Datos y Eliminación de Filas (`lock_piece` y `clear_lines`)

- **`lock_piece()`**: Transfiere de forma permanente los descriptores cromáticos de la pieza en movimiento hacia las direcciones correspondientes de la matriz global `self.grid`.
- **`clear_lines()`**: Realiza un escaneo de memoria sobre las 20 filas del tablero. Si detecta que una fila tiene todos sus índices ocupados (ninguna celda es `None`), ejecuta una rutina de purga: elimina esa secuencia de datos de la lista, incrementa las variables globales de puntuación y añade una fila vacía con valores `None` en la parte superior del buffer de memoria.

5. Avance de nivel y preguntas

Cuando el contador de líneas eliminadas alcanza el establecido por nivel (10 líneas), el flujo activa la rutina `check_level_progress()`, pausando el temporizador físico del juego.

- La subrutina `ask_math_question()` toma el control del canal de Entrada/Salida y configura la operación aritmética adaptativa según el nivel guardado en el sistema operativo:
 - **Nivel 1**: Operación de suma con operandos del 1 al 9.

- **Nivel 2:** Operación de resta parametrizada para evitar resultados negativos.
- **Nivel 3:** Operación de multiplicación para evaluar tablas básicas.
- **Nivel 4:** Operación de división exacta.
- El sistema queda en un estado de espera asíncrono aguardando la confirmación de la tecla ENTER en el componente `ft.TextField`. Al recibir la señal, valida los datos: si el resultado coincide con el valor esperado en memoria, se ejecuta el callback `on_correct` para desbloquear el siguiente nivel; si hay un error, se deriva el flujo a `on_wrong`, abriendo el menú de fallo del sistema.

Diseño de interfaz y elementos visuales

Paleta Cromática Hexadecimal (Mapeada en Memoria)

- **DARK_BLUE** = "#00001e": Buffer de fondo para la ventana principal de interacción.
- **SKY_BLUE** = "#7dbae6": Identificador visual para etiquetas de texto y títulos de jerarquía.
- **YELLOW** = "#ffd700": Estilo para los componentes interactivos de control de pantallas (botones).
- **RED** = "#ff453a": Indicador de error o interrupción por fallo del sistema.
- **GREEN** = "#006400": Indicador de validación correcta en el módulo de Entrada/Salida.
- **Componentes Visuales:** Cuadrícula de juego dimensionada bajo una estructura física, panel lateral de estado para monitorizar el rendimiento de las variables en tiempo real (puntuación, nivel y previsualización de la cola de bloques) y formularios interactivos planos optimizados para evitar el consumo innecesario de recursos gráficos en el hardware anfitrión.

Manual de instalación y configuración del entorno

Requisitos del Sistema

Para garantizar el correcto despliegue del software y el levantamiento de sus hilos de ejecución, el hardware anfitrión debe cumplir con los siguientes parámetros:

- **Sistemas Operativos Soportados:** Windows, macOS, GNU/Linux o Android.
- **Entorno de Ejecución:** Python versión 3.10 o superior instalado en el PATH del sistema.

- **Dependencia Core:** Librería Flet instalada en el entorno global o virtual de Python.
- **Especificación de Memoria:** Memoria RAM física mínima de 4 GB para soportar el entorno de ejecución y los subprocesos de renderizado.

Configuración del Entorno de Ejecución

1. Descargar el instalador de Python correspondiente a la arquitectura de la CPU desde el repositorio oficial: <https://www.python.org/downloads> (Asegurar la activación de la bandera de entorno **"Add Python to PATH"** durante el proceso).
2. Validar la correcta instalación del intérprete abriendo una consola de comandos y ejecutando:

Bash

```
python --version
```

3. Instalar la capa de abstracción gráfica de Flet utilizando el gestor de paquetes de Python (pip):

Bash

```
pip install flet
```

4. Clonar o descargar el script fuente del juego desde el repositorio oficial y abrir el directorio raíz del proyecto dentro de Visual Studio Code o el editor de código de su preferencia.

Ejecución del Software

Para inicializar el bucle de eventos del juego, abra una terminal posicionada en la carpeta raíz del proyecto y ejecute el siguiente comando de consola:

Bash

```
python tetrisflet.py
```

Control de versiones y flujo de trabajo

El ciclo de desarrollo y la trazabilidad del código fuente se gestionaron rigurosamente mediante el sistema de control de versiones distribuido **Git**.

El repositorio no solo funcionó como un almacenamiento secundario seguro para el código fuente del script, sino que actuó como el punto centralizado para coordinar de forma síncrona el avance del equipo de desarrollo. Se utilizó para registrar el cronograma de actividades, documentar los cambios aplicados a los buffers de datos mediante confirmaciones (*commits*) descriptivas y gestionar las ramificaciones del

software necesarias para validar de forma aislada la estabilidad de la asincronía en el motor de Flet antes de consolidar el proyecto final.

El repositorio del proyecto se encuentra disponible en la siguiente dirección pública de acceso:

ESPERAR

Conclusiones técnicas y buenas prácticas aplicadas

1. Logros Técnicos en la Gestión de Procesos

- **Asincronía Efectiva:** Se logró migrar el software hacia un esquema de concurrencia moderna mediante `asyncio`. Esto solucionó el problema crítico de los bloqueos de Entrada/Salida (*deadlocks*), permitiendo que la interfaz permanezca interactiva y en espera de datos textuales del estudiante sin interrumpir ni colapsar el refresco gráfico general del programa.
- **Eficiencia Energética y de Memoria:** Al adoptar la arquitectura declarativa de Flet, el software dejó de forzar repintados imperativos e innecesarios de píxeles a 60 cuadros por segundo. El sistema operativo ahora administra las actualizaciones de forma reactiva a través de `page.update()`, modificando exclusivamente los bloques de la matriz de memoria que sufrieron cambios reales de estado, optimizando los ciclos de reloj de la CPU.
- **Portabilidad Multiplataforma Real:** El uso estratégico de Flet sobre el motor de Flutter eliminó las dependencias de librerías locales rígidas. El sistema interactúa de forma transparente con las APIs nativas de múltiples sistemas operativos, posibilitando que el mismo código fuente en Python corra de manera nativa tanto en sistemas de ventanas de escritorio (Windows, Linux, macOS) como en entornos móviles (Android), adaptándose automáticamente al espacio de pantalla vertical del dispositivo.

2. Integración de Componentes y Calidad de Código

- **Separación Estricta de Funciones:** El diseño lógico aisló por completo las rutinas gráficas de presentación de los algoritmos puros de control matemático del tablero. Esto facilita la depuración y permite escalar el software en el futuro sin riesgo de corromper las variables físicas del juego.
- **Módulo Educativo Sincronizado:** La integración del módulo de preguntas matemáticas se acopló de manera cohesiva con el control de las interrupciones del juego. El contador de líneas actúa como un disparador directo que suspende las tareas lógicas de caída y cede el control al formulario numérico, asegurando

que el progreso del videojuego esté condicionado estrictamente al rendimiento y validación académica del usuario.

3. Estabilidad del Sistema y Control de Versiones

- **Estándares de Codificación y Parámetros en Memoria:** Se aplicaron rigurosamente las convenciones de formato de Python (snake_case y PascalCase) y se centralizaron las variables inmutables del sistema en constantes globales mapeadas al inicio de la carga del script (GRID_COLS, DARK_BLUE).
- **Control de Versiones y Verificación de Flujo:** El uso integrado de Git bajo la metodología Scrum garantizó la trazabilidad completa del script a lo largo de los ciclos de desarrollo. Las pruebas funcionales manuales validaron la estabilidad del sistema ante las señales de entrada/salida (I/O), comprobando que las subrutinas críticas de colisiones matriciales, rotación de bits y el desvío asíncrono hacia las interrupciones del módulo educativo funcionen sin provocar fallos de segmentación o congelar el sistema operativo anfitrión.

El software cumple de manera óptima con el objetivo de unificar un bucle de eventos interactivo con el procesamiento concurrente de filtros aritméticos adaptativos para entornos escolares primarios.

Argumento final

Los tres proyectos comparten un eje transversal claro —educación mediada por tecnología— que conviene resaltar como hilo conductor del Día UVM: HOME JUMP atiende el aprendizaje de idiomas, Tetris Educativo el desarrollo del pensamiento matemático y Chocolate Web Scraper el fortalecimiento de competencias digitales avanzadas. Esta tríada permite ilustrar la contribución integral de la formación universitaria al ODS 4 desde múltiples niveles educativos (básico, medio y superior), articulada con el ODS 9 como motor de innovación.